

Guia de estudio del repositorio pdf-extactext

Esta guia esta pensada para estudiar el proyecto desde cero. La idea no es memorizar cada linea, sino entender que responsabilidad tiene cada parte y como viaja una peticion dentro de la aplicacion.

1. Que hace este proyecto

El proyecto es una API hecha con FastAPI para trabajar con archivos PDF.

En palabras simples:

- Recibe archivos PDF desde un endpoint.
- Valida que el archivo realmente parezca un PDF.
- Extrae texto del PDF con PyMuPDF.
- Calcula un checksum SHA-256 para identificar el contenido.
- Tiene estructura preparada para guardar documentos y usuarios en MongoDB.
- Expone endpoints para crear, listar, actualizar y borrar datos.
- Tiene tests para comprobar partes importantes del comportamiento.

La aplicacion principal esta en `app/main.py`.

2. Como esta organizado el repositorio

El repositorio mezcla varias carpetas, pero las mas importantes son estas:

```
app/  
  api/  
  application/  
  core/  
  domain/  
  infrastructure/  
  services/  
  static/  
tests/  
docs/  
src/  
.env  
.env.example  
pyproject.toml  
docker-compose.yml  
README.md
```

Una forma facil de pensarlo:

- ``app/``: código principal de la API.
- ``tests/``: pruebas automáticas.
- ``docs/``: diagramas y documentación.
- ``src/``: código más antiguo o auxiliar del extractor.
- ``env``: configuración privada de tu máquina.
- ``env.example``: plantilla segura para que otra persona sepa que variables necesita.
- ``pyproject.toml``: dependencias y configuración del proyecto.
- ``docker-compose.yml``: ayuda a levantar servicios externos como MongoDB.

3. La idea de arquitectura por capas

El proyecto intenta separar responsabilidades. Eso ayuda a que el código no quede todo mezclado en un único archivo gigante.

Las capas principales son:

- API o presentación: recibe requests HTTP y devuelve respuestas.
- Application: coordina casos de uso.
- Domain: contiene entidades y reglas de negocio.
- Infrastructure: conecta con cosas externas, como MongoDB.

Ejemplo con usuarios:

Cliente HTTP

```
-> app/api/v1/user_routes.py
-> app/application/services/user_service.py
-> app/domain/entities/user.py
-> app/infrastructure/repositories/mongo_user_repository.py
-> MongoDB
```

La API no debería saber detalles internos de MongoDB. Para eso están los repositorios.

4. app/main.py

Este archivo crea la aplicación FastAPI.

Responsabilidades principales:

- Crear ``app = FastAPI(...)``.
- Configurar título, versión y modo debug.
- Montar archivos estáticos desde ``app/static``.
- Crear la ruta ``/docs`` con Swagger UI personalizado.
- Crear la ruta raíz ``/``.
- Registrar los routers de usuarios y documentos.

Cuando ejecutas:

```
uv run uvicorn app.main:app --reload
```

Uvicorn busca la variable `app` dentro de `app/main.py` y levanta el servidor.

5. app/core/config.py

Este archivo centraliza la configuracion.

Usa `BaseSettings` de Pydantic para leer variables de entorno desde `.env`.

Variables que espera:

```
API_V1_STR=/api/v1
DEBUG=True
DATABASE_URL=mongodb://localhost:27017
DB_NAME=pdf_extractor_db
SECRET_KEY=desarrollo_secreto_utn_2026
```

Que significa cada una:

- `API\_V1\_STR`: version o prefijo de la API.
- `DEBUG`: activa o desactiva modo debug. Debe ser `True` o `False`.
- `DATABASE\_URL`: direccion donde esta MongoDB.
- `DB\_NAME`: nombre de la base de datos.
- `SECRET\_KEY`: clave secreta de la aplicacion.

Importante: `.env` no se sube a GitHub porque puede tener datos sensibles.
Por eso existe `.env.example`, que muestra la forma esperada sin secretos reales.

6. app/api/v1/pdf_router.py

Este archivo define endpoints relacionados con documentos PDF.

Endpoint principal:

```
POST /api/v1/upload
```

Flujo de subida:

1. FastAPI recibe un archivo con `UploadFile`.
2. Se valida que el nombre termine en `.pdf`.

3. Se lee el contenido binario del archivo.
4. ``PDFService.validate_pdf_content(...)`` valida reglas del archivo.
5. Se calcula el checksum con SHA-256.
6. Se extrae texto con PyMuPDF.
7. Si no hay texto extraible, devuelve error 400.
8. Si todo sale bien, devuelve nombre, checksum y preview del texto.

Tambien tiene endpoints CRUD para documentos:

- ``GET /api/v1/documents``: lista documentos.
- ``GET /api/v1/documents/{document_id}``: busca por ID.
- ``GET /api/v1/documents/checksum/{checksum}``: busca por checksum.
- ``PATCH /api/v1/documents/{document_id}``: actualiza filename.
- ``DELETE /api/v1/documents/{document_id}``: borra un documento.

CRUD significa Create, Read, Update, Delete.

7. app/services/pdf_service.py

Este servicio contiene logica relacionada con PDFs.

Funciones importantes:

- ``get_checksum(content)``: calcula un hash SHA-256.
- ``has_valid_pdf_signature(content)``: revisa que el archivo empiece con ``%PDF``.
- ``validate_pdf_content(content)``: aplica reglas de validacion.
- ``extract_text(content)``: abre el PDF y extrae texto.

Reglas de validacion actuales:

- El archivo no puede estar vacio.
- El archivo no puede pesar mas de 5 MB.
- El contenido debe tener firma de PDF.

Por que es importante validar firma:

Un usuario podria subir una imagen llamada ``foto.pdf``. Si solo miramos el nombre, el sistema creeria que es PDF. La firma ``%PDF`` ayuda a detectar si el contenido parece PDF de verdad.

8. app/api/v1/user_routes.py

Este archivo define endpoints de usuarios.

Endpoints principales:

- ``POST /api/v1/users/``: crear usuario.
- ``GET /api/v1/users/{user_id}``: obtener usuario por ID.
- ``GET /api/v1/users/``: listar usuarios.
- ``PUT /api/v1/users/{user_id}``: actualizar usuario.
- ``PATCH /api/v1/users/{user_id}/deactivate``: desactivar usuario.
- ``DELETE /api/v1/users/{user_id}``: eliminar usuario.

Este archivo no debería tener toda la logica de negocio. Su trabajo es recibir datos, llamar a ``UserService`` y transformar errores en respuestas HTTP.

9. app/application/dto/user_dto.py

DTO significa Data Transfer Object.

Son clases que definen la forma de los datos que entran y salen por la API.

Ejemplos:

- ``UserCreateRequest``: datos necesarios para crear usuario.
- ``UserUpdateRequest``: datos necesarios para actualizar usuario.
- ``UserResponse``: datos que se devuelven al cliente.

Pydantic valida automaticamente cosas como:

- Que el email tenga formato de email.
- Que ``full_name`` no este vacio.
- Que el largo del nombre no supere el limite definido.

10. app/application/services/user_service.py

Este archivo representa la capa de aplicacion para usuarios.

Responsabilidades:

- Crear usuarios.
- Buscar usuarios.
- Listar usuarios.
- Actualizar perfiles.
- Desactivar usuarios.
- Eliminar usuarios.

Ejemplo importante:

Antes de crear un usuario, revisa si ya existe otro con el mismo email. Si existe, lanza una ``ValidationException``.

Esto es logica de aplicacion, porque coordina entidades y repositorios.

11. app/domain/entities

La carpeta `domain/entities` contiene las entidades principales del negocio.

User

Archivo:

app/domain/entities/user.py

Representa un usuario.

Campos:

- `id`
- `email`
- `full\_name`
- `is\_active`
- `created\_at`

Tiene metodos como:

- `activate()`
- `deactivate()`
- `update\_profile(full\_name)`

Document

Archivo:

app/domain/entities/document.py

Representa un documento PDF.

Campos:

- `id`
- `filename`
- `checksum`
- `extracted\_text`
- `created\_at`

Tiene `update\_filename(...)`, que valida que el nombre no quede vacio.

12. app/domain/repositories

Esta carpeta define interfaces o contratos.

Un contrato dice: "cualquier repositorio de documentos debe tener estos metodos".

Ejemplo conceptual:

```
DocumentRepository
- find_by_id
- find_by_checksum
- find_all
- create
- update
- delete
```

La ventaja es que el dominio no queda atado a MongoDB. Si mañana se usa PostgreSQL, se podría crear otro repositorio que respete el mismo contrato.

13. app/infrastructure/database

Esta carpeta tiene código relacionado con MongoDB y Pydantic schemas.

connection.py

Se conecta a MongoDB usando:

```
AsyncIOMotorClient(settings.DATABASE_URL)
```

Motor es el driver asincrónico de MongoDB para Python.

Nota importante: actualmente el archivo selecciona ``client.pdf_extractor_db``.

Sería más flexible usar ``client[settings.DB_NAME]``, porque así respeta la variable ``DB_NAME`` del ``.env``.

schemas

Los schemas de infraestructura representan como se validan los datos que van o vienen de MongoDB.

Archivos:

- ``document_schema.py``
- ``user_schema.py``

Estos schemas validan cosas como:

- ``filename`` no vacío.
- ``checksum`` de 64 caracteres.
- ``email`` con formato correcto.
- IDs compatibles con MongoDB.

14. app/infrastructure/repositories

Aquí están las implementaciones concretas de los repositorios usando MongoDB.

Ejemplo:

`MongoDocumentRepository`

Hace operaciones reales contra la colección ``documents``.

Métodos importantes:

- ``find_by_id``: busca por ``_id``.
- ``find_by_checksum``: busca por checksum.
- ``find_all``: lista todos.
- ``create``: inserta un documento.
- ``update``: modifica el filename.
- ``delete``: borra por ID.

También convierte entre dos mundos:

- Schema de base de datos (``DocumentInDB``).
- Entidad de dominio (``Document``).

Esa conversión es importante para que el dominio no dependa de MongoDB.

15. app/api/dependencies.py

Este archivo arma dependencias para FastAPI.

FastAPI tiene un sistema llamado dependency injection.

En simple:

En vez de crear manualmente un repositorio en cada endpoint, FastAPI lo inyecta con ``Depends(...)``.

Ejemplo:

```
document_repository: DocumentRepository = Depends(get_document_repository)
```

Eso significa:

"Cuando alguien llame este endpoint, dame un repositorio de documentos ya preparado".

16. `app/core/exceptions.py` y `domain/exceptions`

Estos archivos contienen errores propios del proyecto.

Ejemplos:

- ``ValidationException``
- ``NotFoundException``
- ``DocumentNotFoundError``
- ``DocumentAlreadyExistsError``

Sirven para no depender solamente de errores genericos de Python. Un error propio explica mejor que paso dentro del negocio.

17. `app/static/custom.css`

Este archivo modifica visualmente Swagger UI.

Swagger UI es la pantalla de documentacion interactiva que ves en:

```
http://127.0.0.1:8000/docs
```

No afecta la logica del backend. Solo cambia estilos visuales.

18. tests

La carpeta ``tests`` contiene pruebas automaticas.

Estructura:

- ``tests/domain``: pruebas de reglas de dominio o servicios puros.

- ``tests/presentation``: pruebas de endpoints HTTP.
- ``tests/data``: pruebas relacionadas con datos y MongoDB.

Ejemplos:

- ``test_document_upload.py``: prueba subir PDFs y archivos invalidos.
- ``test_file_signature_validator.py``: prueba la firma ``%PDF``.
- ``test_checksum.py``: prueba checksum y extraccion.
- ``test_mongo_real.py``: prueba MongoDB real en ``localhost:27017``.

Cuando los tests de Mongo fallan por conexion, normalmente significa que MongoDB no esta levantado, no que la logica de PDF este mal.

19. pyproject.toml

Este archivo declara el proyecto y sus dependencias.

Algunas dependencias importantes:

- ``fastapi``: framework web.
- ``uvicorn``: servidor para correr FastAPI.
- ``pydantic``: validacion de datos.
- ``pydantic-settings``: lectura de variables de entorno.
- ``motor``: conexion asincronica con MongoDB.
- ``pymongo``: herramientas de MongoDB.
- ``pymupdf``: lectura y extraccion de texto de PDFs.
- ``pytest``: tests.

``uv.lock`` guarda versiones exactas para que el entorno sea reproducible.

20. docker-compose.yml

Este archivo ayuda a levantar servicios con Docker.

Tiene un servicio de MongoDB llamado ``basededatos``.

Mongo expone el puerto:

`27017:27017`

Eso permite que la app se conecte con:

`DATABASE_URL=mongodb://localhost:27017`

21. Flujo completo de subida de PDF

Este es el recorrido mental mas importante:

Usuario sube archivo en Swagger

- > POST /api/v1/upload
- > pdf_router.py recibe UploadFile
- > valida extension .pdf
- > lee bytes del archivo
- > PDFService.validate_pdf_content
- > PDFService.get_checksum
- > PDFService.extract_text
- > devuelve JSON con preview

Si algo sale mal:

- Extension incorrecta: error 400.
- Archivo vacio: error 400.
- Archivo demasiado grande: error 400.
- Contenido que no es PDF: error 400.
- PDF sin texto extraible: error 400.

22. Como estudiar este proyecto sin perderse

Orden recomendado:

1. Leer `app/main.py`.
2. Leer `app/api/v1/pdf\_router.py`.
3. Leer `app/services/pdf\_service.py`.
4. Leer `tests/presentation/test\_document\_upload.py`.
5. Leer `app/api/v1/user\_routes.py`.
6. Leer `app/application/services/user\_service.py`.
7. Leer entidades en `app/domain/entities`.
8. Leer repositorios en `app/infrastructure/repositories`.
9. Leer schemas en `app/infrastructure/database/schemas`.
10. Leer configuracion en `.env`, `.env.example` y `app/core/config.py`.

No hace falta entender todo de una sola vez. Primero entiende el camino del request:

Ruta -> Servicio -> Entidad/Repositorio -> Respuesta

Con eso ya tenes la mitad del mapa mental.

23. Mini glosario

- API: interfaz para que otros programas hablen con tu aplicacion.
- Endpoint: una URL especifica de la API.

- Request: lo que el cliente envia.
- Response: lo que la API devuelve.
- Router: archivo que agrupa endpoints.
- DTO: objeto que define datos de entrada o salida.
- Entidad: objeto principal del dominio.
- Repositorio: objeto encargado de guardar o buscar datos.
- Schema: modelo de validacion de datos.
- MongoDB: base de datos NoSQL orientada a documentos.
- Checksum: hash que identifica el contenido de un archivo.
- SHA-256: algoritmo para calcular un hash de 64 caracteres.
- Variable de entorno: configuracion externa al codigo.
- ``.env``: archivo local con variables de entorno.
- Test: prueba automatica para verificar comportamiento.

24. Comandos utiles

Instalar dependencias:

```
uv sync
```

Levantar la API:

```
uv run uvicorn app.main:app --reload
```

Abrir documentacion:

```
http://127.0.0.1:8000/docs
```

Correr tests:

```
uv run pytest
```

Levantar MongoDB con Docker:

```
docker run -d -p 27017:27017 --name mongo mongo:7
```

25. Ideas clave para explicar en una defensa

Si tenes que explicar el proyecto, podes decir:

"La aplicacion usa FastAPI para exponer endpoints HTTP. La configuracion se lee desde variables de entorno con Pydantic Settings. La logica de PDFs esta

separada en un servicio llamado `PDFService`, que valida el archivo, calcula checksum y extrae texto. Los usuarios y documentos estan modelados como entidades de dominio. La persistencia esta abstraída con repositorios, y la implementacion actual usa MongoDB mediante Motor. Los tests verifican validaciones de subida, checksum, firma de archivo y comportamiento con MongoDB."

Eso resume bastante bien la arquitectura sin entrar en detalles innecesarios.